Question 3: Memory Management

1. Introduction

Memory management is an operating system function that controls how memory is allocated to processes and released when processes finish. This program simulates memory allocation and deallocation using two allocation algorithms: First-Fit and Best-Fit The purpose of the simulation is to observe how the two algorithms allocate memory and compare their memory utilization and search effort under different memory capacities and process sizes.

2. Objectives

- Simulate memory allocation to processes.
- Simulate deallocation when a process finishes.
- Implement First-Fit and Best-Fit allocation algorithms.
- Test both algorithms with different memory capacities and process sizes.
- Compare memory utilization and allocation search effort.

3. Programming language, tools and operating system

- **Programming language:** Python 3
- **Operating system:** Parrot OS
- **Editor:** kate
- **Execution command:** `python3 memory_management.py`

4. Allocation algorithms

4.1 First-Fit

First-Fit searches the available memory blocks from the beginning. It allocates a process to the first free block that is large enough to satisfy the request. It stops searching once a suitable block is found.

4.2 Best-Fit

Best-Fit searches the available free blocks and chooses the smallest block that is large enough for the process. It checks all available blocks before making its choice.

5. Program implementation

The program represents memory as a list of blocks. Each block stores its size and either the name of the process using it or an indication thatit is free.

The program contains functions for:

- Creating the initial free memory block.

- Allocating memory using the selected algorithm.
- Deallocating memory used by a process.
- Merging neighbouring free blocks after deallocation.
- Calculating memory utilization and counting the blocks examined during allocation.
- Running test scenarios and displaying a comparison.

memory_management.py

home › potato47 › Documents › Tools › OS Assignments › Memory_Management › memory_management.py

```python
# ========================================
# QUESTION 3: MEMORY MANAGEMENT
# Algorithms: First-Fit and Best-Fit
# ========================================


# 1. Create the initial memory
def create_memory(total_memory):
    return [{"size": total_memory, "process": None}]


# 2. Allocate memory to a process
def allocate(memory, process, request, algorithm, stats):

    selected = -1

    # Search for a suitable free memory block
    for i in range(len(memory)):

        stats["checked"] += 1

        if memory[i]["process"] is None and memory[i]["size"] >= request:

            # First-Fit chooses the first suitable block
            if algorithm == "First-Fit":
                selected = i
                break

            # Best-Fit chooses the smallest suitable block
            elif selected == -1 or memory[i]["size"] < memory[selected]["size"]:
                selected = i

    # Check whether a suitable block was found
    if selected == -1:
        stats["failed"] += 1
        return

    # Calculate the memory remaining after allocation
    remaining = memory[selected]["size"] - request

    # Allocate the requested memory
    memory[selected]["size"] = request
    memory[selected]["process"] = process

    # Create a new free block if memory remains
    if remaining > 0:
        memory.insert(selected + 1, {
            "size": remaining,
            "process": None
        })
```

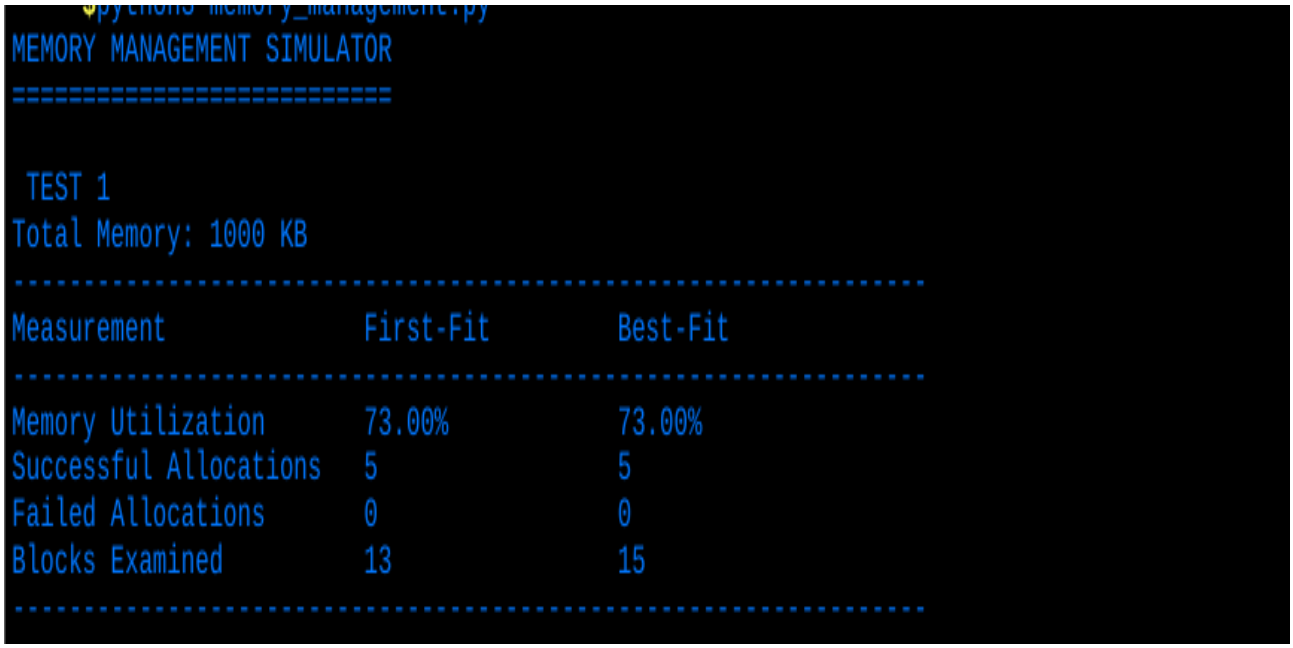
Capture the main parts of `memory_management.py`, especially the First-Fit and Best-Fit selection logic, allocation/deallocation functions, and test scenarios. If the code does not fit clearly in one screenshot, use two or three numbered screenshots.

6. Testing procedure

Both algorithms were tested using the same process requests within each test. This makes the comparison fair because the algorithms receive the same memory capacity and workload.

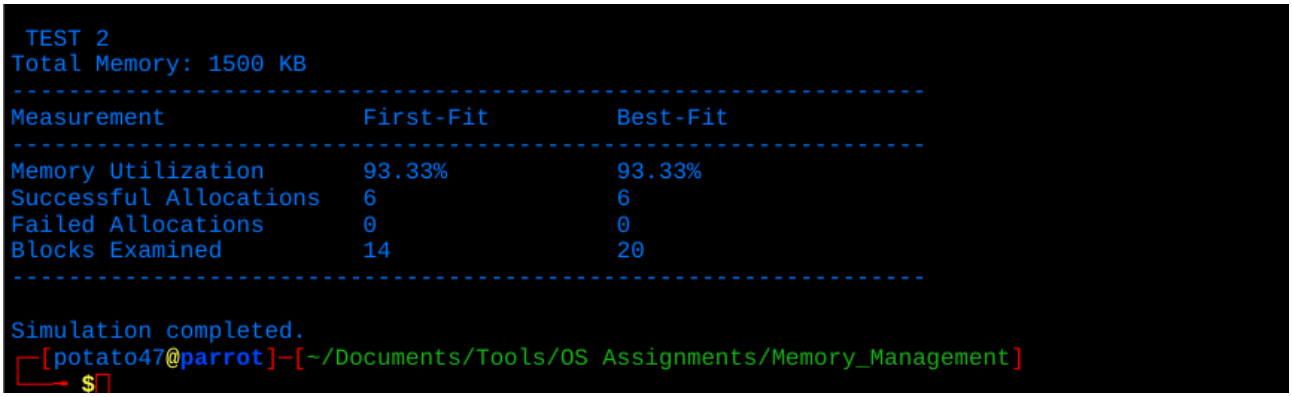
Test 1

- Total memory: **1,000 KB**
- Process sizes and operations: Test 1 workload defined in the source code.
- Algorithms tested: First-Fit and Best-Fit.
-



Test 2

- Total memory: **1,500 KB**
- Process sizes and operations: Test 2 workload defined in the source code.
- Algorithms tested: First-Fit and Best-Fit.



.

7. Results and comparison

The following table records the output produced by the program.

Measurement Test 1 Test 1 Test 2 Test 2
First-Fit Best-Fit First-Fit Best-Fit

Total memory 1,000 KB 1,000 KB 1,500 KB 1,500 KB

Memory 73.00% 73.00% 93.33% 93.33%
utilization

Successful 5 5 6 6
allocations

Failed 0 0 0 0
allocations

Blocks 13 15 14 20
examined

7.1 Meaning of the measurements

- **Memory utilization:** The percentage of total memory occupied by allocated processes at the end of a test.
- **Successful allocations:** The number of process memory requests that were allocated successfully.
- **Failed allocations:** The number of process memory requests that could not be allocated.
- **Blocks examined:** The total number of memory blocks checked while searching for a suitable block during allocation. This is a simple measure of search effort, not a direct measurement of execution time.

Memory utilization is calculated using:

```
Memory utilization = (Allocated memory / Total memory) × 100
```

8. Discussion of results

In Test 1, both algorithms achieved 73.00% memory utilization. They each completed five allocations and had no failed allocations. First-Fit examined 13 blocks, while Best-Fit examined 15 blocks.

In Test 2, both algorithms achieved 93.33% memory utilization. They each completed six allocations and had no failed allocations. First-Fit examined 14 blocks, while Best-Fit examined 20 blocks.

For these two workloads, both algorithms achieved the same final memory utilization and allocation success. First-Fit examined fewer blocks than Best-Fit in both tests. This indicates that First-Fit required less search effort for the workloads used in this simulation. These results do not prove that First-Fit is always faster or better. The results depend on the memory sizes and order of process requests used in a test. The blocks-examined measurement is a simple indicator of search work; actual execution time was not measured.

9. Conclusion

The program successfully simulated memory allocation and deallocation using First-Fit and Best-Fit. Both algorithms were tested with two different memory capacities and sets of process sizes. In the tests performed, both algorithms achieved the same memory utilization and allocation success. First-Fit examined fewer memory blocks than Best-Fit in both test scenarios. This demonstrates that different allocation strategies can require different amounts of search effort even when they produce the same final memory utilization. The experiment shows that the performance of a memory allocation algorithm depends on the workload being tested. No conclusion that one algorithm is always better can be made from these two tests alone.

10. References

1. Silberschatz, A., Galvin, P. B., and Gagne, G. *Operating System Concepts*.
2. Arpaci-Dusseau, R. H., and Arpaci-Dusseau, A. C. *Operating Systems: Three Easy Pieces*. See the memory virtualization material. Available at: https://pages.cs.wisc.edu/\~remzi/OSTEP/